

Клиент ТИМП Какулин

Создано системой Doxygen 1.17.0

1 Иерархический список классов	1
1.1 Иерархия классов	1
2 Алфавитный указатель классов	3
2.1 Классы	3
3 Список файлов	5
3.1 Файлы	5
4 Классы	7
4.1 Класс ArgsDecodeException	7
4.1.1 Подробное описание	8
4.1.2 Конструктор(ы)	8
4.1.2.1 ArgsDecodeException()	8
4.2 Класс AuthException	8
4.2.1 Подробное описание	9
4.2.2 Конструктор(ы)	9
4.2.2.1 AuthException()	9
4.3 Класс BasicClientException	10
4.3.1 Подробное описание	10
4.3.2 Конструктор(ы)	10
4.3.2.1 BasicClientException()	10
4.3.3 Методы	11
4.3.3.1 what()	11
4.4 Класс DataDecodeException	11
4.4.1 Подробное описание	12
4.4.2 Конструктор(ы)	12
4.4.2.1 DataDecodeException()	12
4.5 Класс InputOutputException	13
4.5.1 Подробное описание	13
4.5.2 Конструктор(ы)	14
4.5.2.1 InputOutputException()	14
4.6 Класс InputOutputManager	14
4.7 Класс MyTestReporter	14
4.7.1 Подробное описание	15
4.7.2 Методы	15
4.7.2.1 ReportSummary()	15
4.7.2.2 ReportTestFinish()	15
4.7.2.3 ReportTestStart()	16
4.8 Класс NetworkException	16
4.8.1 Подробное описание	17
4.8.2 Конструктор(ы)	17
4.8.2.1 NetworkException()	17
4.9 Класс NetworkManager	17

4.9.1 Подробное описание	17
4.10 Класс <code>UserInterface</code>	18
4.10.1 Подробное описание	18
4.10.2 Конструктор(ы)	18
4.10.2.1 <code>UserInterface()</code>	18
4.10.3 Методы	19
4.10.3.1 <code>getAddress()</code>	19
4.10.3.2 <code>getConfigFilePath()</code>	19
4.10.3.3 <code>getInputFilePath()</code>	19
4.10.3.4 <code>getOutputFilePath()</code>	19
4.10.3.5 <code>getPort()</code>	19
5 Файлы	21
5.1 Файл <code>client/source/main.cpp</code>	21
5.1.1 Подробное описание	21
5.1.2 Функции	22
5.1.2.1 <code>main()</code>	22
5.2 Файл <code>client/source/modules/exceptions.h</code>	22
5.2.1 Подробное описание	23
5.3 <code>exceptions.h</code>	23
5.4 Файл <code>client/source/modules/io.h</code>	24
5.4.1 Подробное описание	24
5.5 <code>io.h</code>	24
5.6 Файл <code>client/source/modules/network.h</code>	25
5.6.1 Подробное описание	25
5.7 <code>network.h</code>	26
5.8 Файл <code>client/source/modules/ui.h</code>	26
5.8.1 Подробное описание	26
5.9 <code>ui.h</code>	27
Предметный указатель	29

Глава 1

Иерархический список классов

1.1 Иерархия классов

Иерархия классов.

std::exception	
BasicClientException	10
ArgsDecodeException	7
AuthException	8
DataDecodeException	11
InputOutputException	13
NetworkException	16
InputOutputManager	14
NetworkManager	17
UnitTest::TestReporter	
MyTestReporter	14
UserInterface	18

Глава 2

Алфавитный указатель классов

2.1 Классы

Классы с их кратким описанием.

ArgsDecodeException	
Исключение для ошибок декодирования аргументов	7
AuthException	
Исключение для ошибок аутентификации	8
BasicClientException	
Базовый класс для клиентских ошибок	10
DataDecodeException	
Исключение для ошибок декодирования данных	11
InputOutputException	
Исключение для ошибок ввода-вывода	13
InputOutputManager	
.	14
MyTestReporter	
Класс логгера для вывода детализированной информации о тестах	14
NetworkException	
Исключение для сетевых ошибок	16
NetworkManager	
Класс для управления сетевым подключением и взаимодействием	17
UserInterface	
Класс для управления пользовательским интерфейсом	18

Глава 3

Список файлов

3.1 Файлы

Полный список документированных файлов.

client/source/ main.cpp	
Главный файл программы	21
client/source/modules/ exceptions.h	
Определения классов исключений для клиентской части	22
client/source/modules/ io.h	
Определения классов для управления вводом-выводом	24
client/source/modules/ network.h	
Определения классов для управления сетевым взаимодействием	25
client/source/modules/ ui.h	
Определения классов для пользовательского интерфейса	26

Глава 4

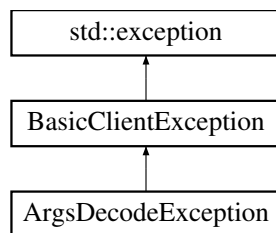
Классы

4.1 Класс ArgsDecodeException

Исключение для ошибок декодирования аргументов.

```
#include <exceptions.h>
```

Граф наследования: ArgsDecodeException:



Открытые члены

- [ArgsDecodeException](#) (const std::string &message, const std::string &func)
Конструктор [ArgsDecodeException](#).

Открытые члены унаследованные от [BasicClientException](#)

- [BasicClientException](#) (const std::string &name, const std::string &message, const std::string &func)
Конструктор [BasicClientException](#).
- const char * [what](#) () const noexcept override
Метод для получения сообщения об ошибке.

Дополнительные унаследованные члены

Защищенные данные унаследованные от [BasicClientException](#)

- std::string name
Имя исключения.
- std::string func
Имя функции, в которой возникла ошибка.
- std::string message
Сообщение об ошибке.

4.1.1 Подробное описание

Исключение для ошибок декодирования аргументов.

4.1.2 Конструктор(ы)

4.1.2.1 ArgsDecodeException()

```
ArgsDecodeException::ArgsDecodeException (
    const std::string & message,
    const std::string & func)
```

Конструктор [ArgsDecodeException](#).

Аргументы

message	Сообщение об ошибке.
func	Имя функции, в которой возникла ошибка.

Объявления и описания членов классов находятся в файлах:

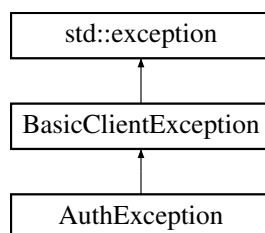
- [client/source/modules/exceptions.h](#)
- [client/source/modules/exceptions.cpp](#)

4.2 Класс AuthException

Исключение для ошибок аутентификации.

```
#include <exceptions.h>
```

Граф наследования: AuthException:



Открытые члены

- [AuthException](#) (const std::string &message, const std::string &func)
Конструктор [AuthException](#).

Открытые члены унаследованные от BasicClientException

- [BasicClientException](#) (const std::string &name, const std::string &message, const std::string &func)
Конструктор [BasicClientException](#).
- const char * [what](#) () const noexcept override
Метод для получения сообщения об ошибке.

Дополнительные унаследованные члены

Защищенные данные унаследованные от BasicClientException

- std::string name
Имя исключения.
- std::string func
Имя функции, в которой возникла ошибка.
- std::string message
Сообщение об ошибке.

4.2.1 Подробное описание

Исключение для ошибок аутентификации.

4.2.2 Конструктор(ы)

4.2.2.1 AuthException()

```
AuthException::AuthException (  
    const std::string & message,  
    const std::string & func)
```

Конструктор [AuthException](#).

Аргументы

message	Сообщение об ошибке.
func	Имя функции, в которой возникла ошибка.

Объявления и описания членов классов находятся в файлах:

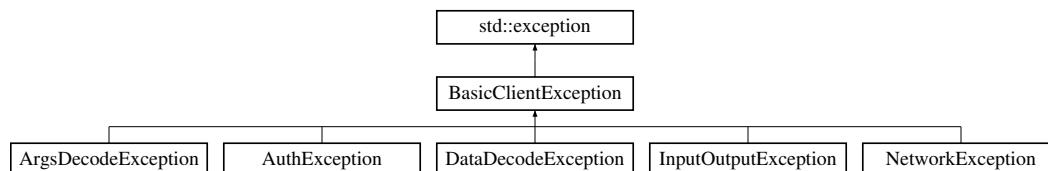
- client/source/modules/[exceptions.h](#)
- client/source/modules/[exceptions.cpp](#)

4.3 Класс BasicClientException

Базовый класс для клиентских ошибок.

```
#include <exceptions.h>
```

Граф наследования: BasicClientException:



Открытые члены

- [BasicClientException](#) (const std::string &name, const std::string &message, const std::string &func)
Конструктор [BasicClientException](#).
- const char * [what](#) () const noexcept override
Метод для получения сообщения об ошибке.

Защищенные данные

- std::string name
Имя исключения.
- std::string func
Имя функции, в которой возникла ошибка.
- std::string message
Сообщение об ошибке.

4.3.1 Подробное описание

Базовый класс для клиентских ошибок.

4.3.2 Конструктор(ы)

4.3.2.1 BasicClientException()

```
BasicClientException::BasicClientException (
    const std::string & name,
    const std::string & message,
    const std::string & func)
```

Конструктор [BasicClientException](#).

Аргументы

name	Имя исключения.
message	Сообщение об ошибке.
func	Имя функции, в которой возникла ошибка.

4.3.3 Методы

4.3.3.1 what()

```
const char * BasicClientException::what () const [override], [noexcept]
```

Метод для получения сообщения об ошибке.

Возвращает

Сообщение об ошибке.

Объявления и описания членов классов находятся в файлах:

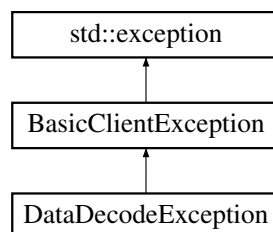
- [client/source/modules/exceptions.h](#)
- [client/source/modules/exceptions.cpp](#)

4.4 Класс DataDecodeException

Исключение для ошибок декодирования данных.

```
#include <exceptions.h>
```

Граф наследования: DataDecodeException:



Открытые члены

- [DataDecodeException](#) (const std::string &message, const std::string &func)
Конструктор [DataDecodeException](#).

Открытые члены унаследованные от [BasicClientException](#)

- [BasicClientException](#) (const std::string &name, const std::string &message, const std::string &func)
Конструктор [BasicClientException](#).
- const char * [what](#) () const noexcept override
Метод для получения сообщения об ошибке.

Дополнительные унаследованные члены

Защищенные данные унаследованные от [BasicClientException](#)

- std::string name
Имя исключения.
- std::string func
Имя функции, в которой возникла ошибка.
- std::string message
Сообщение об ошибке.

4.4.1 Подробное описание

Исключение для ошибок декодирования данных.

4.4.2 Конструктор(ы)

4.4.2.1 DataDecodeException()

```
DataDecodeException::DataDecodeException (
    const std::string & message,
    const std::string & func)
```

Конструктор [DataDecodeException](#).

Аргументы

message	Сообщение об ошибке.
func	Имя функции, в которой возникла ошибка.

Объявления и описания членов классов находятся в файлах:

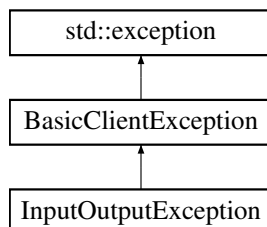
- client/source/modules/[exceptions.h](#)
- client/source/modules/[exceptions.cpp](#)

4.5 Класс InputOutputException

Исключение для ошибок ввода-вывода.

```
#include <exceptions.h>
```

Граф наследования: InputOutputException:



Открытые члены

- [InputOutputException](#) (const std::string &message, const std::string &func)
Конструктор [InputOutputException](#).

Открытые члены унаследованные от [BasicClientException](#)

- [BasicClientException](#) (const std::string &name, const std::string &message, const std::string &func)
Конструктор [BasicClientException](#).
- const char * [what](#) () const noexcept override
Метод для получения сообщения об ошибке.

Дополнительные унаследованные члены

Защищенные данные унаследованные от [BasicClientException](#)

- std::string name
Имя исключения.
- std::string func
Имя функции, в которой возникла ошибка.
- std::string message
Сообщение об ошибке.

4.5.1 Подробное описание

Исключение для ошибок ввода-вывода.

4.5.2 Конструктор(ы)

4.5.2.1 InputOutputException()

```
InputOutputException::InputOutputException (
    const std::string & message,
    const std::string & func)
```

Конструктор [InputOutputException](#).

Аргументы

message	Сообщение об ошибке.
func	Имя функции, в которой возникла ошибка.

Объявления и описания членов классов находятся в файлах:

- [client/source/modules/exceptions.h](#)
- [client/source/modules/exceptions.cpp](#)

4.6 Класс InputOutputManager

Открытые члены

- `InputOutputManager (const std::string &config, const std::string &input, const std::string &output)`
- `std::array< std::string, 2 > conf ()`
- `std::vector< std::vector< uint64_t > > read ()`
- `void write (const std::vector< uint64_t > &data)`

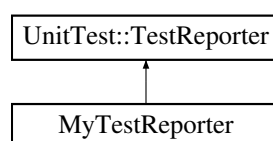
Объявления и описания членов классов находятся в файлах:

- [client/source/modules/io.h](#)
- [client/source/modules/io.cpp](#)

4.7 Класс MyTestReporter

Класс логгера для вывода детализированной информации о тестах.

Граф наследования: MyTestReporter:



Открытые члены

- void [ReportTestStart](#) (UnitTest::TestDetails const &test) override
Сообщает о начале выполнения теста.
- void [ReportTestFinish](#) (UnitTest::TestDetails const &test, float secondsElapsed) override
Сообщает об окончании выполнения теста.
- void ReportFailure (UnitTest::TestDetails const &test, char const *failure) override
- void [ReportSummary](#) (int totalTestCount, int failedTestCount, int failureCount, float secondsElapsed) override
Сообщает об общей сводке выполнения тестов.

4.7.1 Подробное описание

Класс логгера для вывода детализированной информации о тестах.

4.7.2 Методы

4.7.2.1 ReportSummary()

```
void MyTestReporter::ReportSummary (  
    int totalTestCount,  
    int failedTestCount,  
    int failureCount,  
    float secondsElapsed) [inline], [override]
```

Сообщает об общей сводке выполнения тестов.

Аргументы

totalTestCount	Общее количество тестов.
failedTestCount	Количество неудачных тестов.
failureCount	Общее количество сбоев.
secondsElapsed	Общее время выполнения тестов в секундах.

4.7.2.2 ReportTestFinish()

```
void MyTestReporter::ReportTestFinish (  
    UnitTest::TestDetails const & test,  
    float secondsElapsed) [inline], [override]
```

Сообщает об окончании выполнения теста.

Аргументы

test	Детали теста.
secondsElapsed	Время выполнения теста в секундах.

4.7.2.3 ReportTestStart()

```
void MyTestReporter::ReportTestStart (
    UnitTest::TestDetails const & test) [inline], [override]
```

Сообщает о начале выполнения теста.

Аргументы

test	Детали теста.
------	---------------

Объявления и описания членов класса находятся в файле:

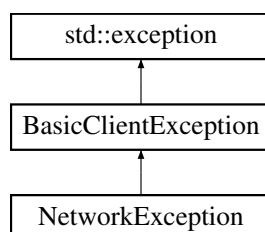
- `unit/source/main.cpp`

4.8 Класс NetworkException

Исключение для сетевых ошибок.

```
#include <exceptions.h>
```

Граф наследования: NetworkException:



Открытые члены

- [NetworkException](#) (const std::string &message, const std::string &func)
Конструктор [NetworkException](#).

Открытые члены унаследованные от [BasicClientException](#)

- [BasicClientException](#) (const std::string &name, const std::string &message, const std::string &func)
Конструктор [BasicClientException](#).
- const char * [what](#) () const noexcept override
Метод для получения сообщения об ошибке.

Дополнительные унаследованные члены

Защищенные данные унаследованные от [BasicClientException](#)

- std::string name
Имя исключения.
- std::string func
Имя функции, в которой возникла ошибка.
- std::string message
Сообщение об ошибке.

4.8.1 Подробное описание

Исключение для сетевых ошибок.

4.8.2 Конструктор(ы)

4.8.2.1 NetworkException()

```
NetworkException::NetworkException (
    const std::string & message,
    const std::string & func)
```

Конструктор [NetworkException](#).

Аргументы

message	Сообщение об ошибке.
func	Имя функции, в которой возникла ошибка.

Объявления и описания членов классов находятся в файлах:

- [client/source/modules/exceptions.h](#)
- [client/source/modules/exceptions.cpp](#)

4.9 Класс NetworkManager

Класс для управления сетевым подключением и взаимодействием.

```
#include <network.h>
```

Открытые члены

- NetworkManager (const std::string &address, uint16_t port)
- std::string & getAddress ()
- uint16_t & getPort ()
- void conn ()
- void auth (const std::string &username, const std::string &password)
- std::vector< uint64_t > calc (const std::vector< std::vector< uint64_t > > &data)
- void close ()

4.9.1 Подробное описание

Класс для управления сетевым подключением и взаимодействием.

Объявления и описания членов классов находятся в файлах:

- [client/source/modules/network.h](#)
- [client/source/modules/network.cpp](#)

4.10 Класс `UserInterface`

Класс для управления пользовательским интерфейсом.

```
#include <ui.h>
```

Открытые члены

- `UserInterface` (int argc, char *argv[])
Конструктор класса `UserInterface`.
- `~UserInterface` ()
Деструктор класса `UserInterface`.
- `std::string & getAddress` ()
Метод для получения адреса.
- `uint16_t & getPort` ()
Метод для получения порта.
- `std::string & getInputFilePath` ()
Метод для получения пути к входному файлу.
- `std::string & getOutputFilePath` ()
Метод для получения пути к выходному файлу.
- `std::string & getConfigFilePath` ()
Метод для получения пути к конфигурационному файлу.
- `void run` ()
Метод для запуска интерфейса.

4.10.1 Подробное описание

Класс для управления пользовательским интерфейсом.

4.10.2 Конструктор(ы)

4.10.2.1 `UserInterface()`

```
UserInterface::UserInterface (
    int argc,
    char * argv[])
```

Конструктор класса `UserInterface`.

Аргументы

argc	Количество аргументов командной строки.
argv	Аргументы командной строки.

Исключения

<code>ArgsDecodeException</code>	Если отсутствуют обязательные параметры или их значения.
----------------------------------	--

4.10.3 Методы

4.10.3.1 `getAddress()`

`std::string & UIInterface::getAddress ()`

Метод для получения адреса.

Возвращает

Адрес.

4.10.3.2 `getConfigFilePath()`

`std::string & UIInterface::getConfigFilePath ()`

Метод для получения пути к конфигурационному файлу.

Возвращает

Путь к конфигурационному файлу.

4.10.3.3 `getInputFilePath()`

`std::string & UIInterface::getInputFilePath ()`

Метод для получения пути к входному файлу.

Возвращает

Путь к входному файлу.

4.10.3.4 `getOutputFilePath()`

`std::string & UIInterface::getOutputFilePath ()`

Метод для получения пути к выходному файлу.

Возвращает

Путь к выходному файлу.

4.10.3.5 `getPort()`

`uint16_t & UIInterface::getPort ()`

Метод для получения порта.

Возвращает

Порт.

Объявления и описания членов классов находятся в файлах:

- `client/source/modules/ui.h`
- `client/source/modules/ui.cpp`

Глава 5

Файлы

5.1 Файл client/source/main.cpp

Главный файл программы.

```
#include "modules/ui.h"  
#include <iostream>
```

Функции

- int `main` (int argc, char *argv[])
Главная функция программы.

5.1.1 Подробное описание

Главный файл программы.

Этот файл содержит функцию `main`, которая инициализирует интерфейс пользователя и запускает программу.

Дата

25.06.2026

Версия

1.0

Авторы

Какулин А.А.

Авторство

ИБСТ ПГУ

5.1.2 Функции

5.1.2.1 main()

```
int main (  
    int argc,  
    char * argv[])
```

Главная функция программы.

Инициализирует объект [UserInterface](#) и запускает его. Обработывает все исключения, возникающие во время выполнения программы.

Аргументы

argc	Количество аргументов командной строки.
argv	Аргументы командной строки.

Возвращает

Код завершения программы. 0 - успешное завершение, 1 - ошибка.

5.2 Файл client/source/modules/exceptions.h

Определения классов исключений для клиентской части.

```
#include <exception>  
#include <string>
```

Классы

- class [BasicClientException](#)
Базовый класс для клиентских ошибок.
- class [InputOutputException](#)
Исключение для ошибок ввода-вывода.
- class [ArgsDecodeException](#)
Исключение для ошибок декодирования аргументов.
- class [DataDecodeException](#)
Исключение для ошибок декодирования данных.
- class [AuthException](#)
Исключение для ошибок аутентификации.
- class [NetworkException](#)
Исключение для сетевых ошибок.

5.2.1 Подробное описание

Определения классов исключений для клиентской части.

Этот файл содержит определения классов для обработки различных исключений, возникающих в клиентской части программы.

Дата

25.06.2026

Версия

1.0

Авторы

Какулин А.А.

Авторство

ИБСТ ПГУ

5.3 exceptions.h

[См. документацию.](#)

```
00001 #ifndef ERRORS_H
00002 #define ERRORS_H
00003
00004 #include <exception>
00005 #include <string>
00006
00019 class BasicClientException : public std::exception
00020 {
00021 public:
00028     BasicClientException(const std::string &name, const std::string &message, const std::string &func);
00029
00034     const char *what() const noexcept override;
00035
00036 protected:
00037     std::string name;
00038     std::string func;
00039     mutable std::string message;
00040 };
00041
00045 class InputOutputException : public BasicClientException
00046 {
00047 public:
00053     InputOutputException(const std::string &message, const std::string &func);
00054 };
00055
00059 class ArgsDecodeException : public BasicClientException
00060 {
00061 public:
00067     ArgsDecodeException(const std::string &message, const std::string &func);
00068 };
00069
00073 class DataDecodeException : public BasicClientException
00074 {
00075 public:
00081     DataDecodeException(const std::string &message, const std::string &func);
00082 };
00083
00087 class AuthException : public BasicClientException
00088 {
00089 public:
00095     AuthException(const std::string &message, const std::string &func);
00096 };
00097
00101 class NetworkException : public BasicClientException
00102 {
00103 public:
00109     NetworkException(const std::string &message, const std::string &func);
00110 };
00111
00112 #endif // ERRORS_H
```

5.4 Файл client/source/modules/io.h

Определения классов для управления вводом-выводом.

```
#include <string>
#include <vector>
#include <array>
#include "exceptions.h"
```

Классы

- class [InputOutputManager](#)

5.4.1 Подробное описание

Определения классов для управления вводом-выводом.

Этот файл содержит определения классов для работы с файлами ввода-вывода и конфигурационными данными.

Дата

25.06.2026

Версия

1.0

Авторы

Какулин А.А.

Авторство

ИБСТ ПГУ

5.5 io.h

[См. документацию.](#)

```
00001 #ifndef IO_MANAGER_H
00002 #define IO_MANAGER_H
00003
00004 #include <string>
00005 #include <vector>
00006 #include <array>
00007 #include "exceptions.h"
00008
00018
00019 class InputOutputManager {
00020 public:
00021     InputOutputManager(
00022         const std::string& config,
00023         const std::string& input,
00024         const std::string& output
00025     );
00026
00027     std::array<std::string, 2> conf();
00028
00029     std::vector<std::vector<uint64_t>> read();
00030     void write(const std::vector<uint64_t>& data);
00031
00032 private:
00033     std::string config;
00034     std::string input;
00035     std::string output;
00036 };
00037
00038 #endif // IO_MANAGER_H
```

5.6 Файл client/source/modules/network.h

Определения классов для управления сетевым взаимодействием.

```
#include <string>
#include <vector>
#include <cstdint>
```

Классы

- class [NetworkManager](#)

Класс для управления сетевым подключением и взаимодействием.

5.6.1 Подробное описание

Определения классов для управления сетевым взаимодействием.

Этот файл содержит определения классов для управления сетевыми подключениями и передачей данных.

Дата

25.06.2026

Версия

1.0

Авторы

Какулин А.А.

Авторство

ИБСТ ПГУ

5.7 network.h

[См. документацию.](#)

```

00001 #ifndef NETWORK_MANAGER_H
00002 #define NETWORK_MANAGER_H
00003
00004 #include <string>
00005 #include <vector>
00006 #include <stdint>
00007
00017
00021 class NetworkManager
00022 {
00023 public:
00024     NetworkManager(const std::string &address, uint16_t port);
00025
00026     std::string &getAddress();
00027     uint16_t &getPort();
00028
00029     void conn();
00030     void auth(const std::string &username, const std::string &password);
00031
00032     std::vector<uint64_t> calc(const std::vector<std::vector<uint64_t>> &data);
00033
00034     void close();
00035
00036 private:
00037     int socket;
00038     std::string address;
00039     uint16_t port;
00040 };
00041
00042 #endif // NETWORK_MANAGER_H

```

5.8 Файл client/source/modules/ui.h

Определения классов для пользовательского интерфейса.

```

#include "io.h"
#include "network.h"
#include "exceptions.h"
#include <string>
#include <vector>

```

Классы

- class [UserInterface](#)
Класс для управления пользовательским интерфейсом.

5.8.1 Подробное описание

Определения классов для пользовательского интерфейса.

Этот файл содержит определения классов для обработки пользовательского ввода и управления интерфейсом.

Дата

25.06.2026

Версия

1.0

Авторы

Какулин А.А.

Авторство

ИБСТ ПГУ

5.9 ui.h

[См. документацию.](#)

```
00001 #ifndef UI_H
00002 #define UI_H
00003
00004 #include "io.h"
00005 #include "network.h"
00006 #include "exceptions.h"
00007 #include <string>
00008 #include <vector>
00009
00019
00023 class UserInterface
00024 {
00025 public:
00032     UserInterface(int argc, char *argv[]);
00033
00037     ~UserInterface();
00038
00043     std::string &getAddress();
00044
00049     uint16_t &getPort();
00050
00055     std::string &getInputFilePath();
00056
00061     std::string &getOutputFilePath();
00062
00067     std::string &getConfigFilePath();
00068
00072     void run();
00073
00074 private:
00075     std::string address;
00076     uint16_t port;
00077     std::string input_path;
00078     std::string output_path;
00079     std::string config_path;
00080
00081     InputOutputManager *io_man;
00082     NetworkManager *net_man;
00083
00084     bool help_flag;
00085
00092     void parseArgs(int argc, char *argv[]);
00093
00097     void showHelp();
00098 };
00099
00100 #endif // UI_H
```


Предметный указатель

- ArgsDecodeException, [7](#)
 - ArgsDecodeException, [8](#)
- AuthException, [8](#)
 - AuthException, [9](#)
- BasicClientException, [10](#)
 - BasicClientException, [10](#)
 - what, [11](#)
- client/source/main.cpp, [21](#)
- client/source/modules/exceptions.h, [22](#), [23](#)
- client/source/modules/io.h, [24](#)
- client/source/modules/network.h, [25](#), [26](#)
- client/source/modules/ui.h, [26](#), [27](#)
- DataDecodeException, [11](#)
 - DataDecodeException, [12](#)
- getAddress
 - UserInterface, [19](#)
- getConfigFilePath
 - UserInterface, [19](#)
- getInputFilePath
 - UserInterface, [19](#)
- getOutputFilePath
 - UserInterface, [19](#)
- getPort
 - UserInterface, [19](#)
- InputOutputException, [13](#)
 - InputOutputException, [14](#)
- InputOutputManager, [14](#)
- main
 - main.cpp, [22](#)
- main.cpp
 - main, [22](#)
- MyTestReporter, [14](#)
 - ReportSummary, [15](#)
 - ReportTestFinish, [15](#)
 - ReportTestStart, [15](#)
- NetworkException, [16](#)
 - NetworkException, [17](#)
- NetworkManager, [17](#)
- ReportSummary
 - MyTestReporter, [15](#)
- ReportTestFinish
 - MyTestReporter, [15](#)
- ReportTestStart
 - MyTestReporter, [15](#)
- MyTestReporter, [15](#)
- UserInterface, [18](#)
 - getAddress, [19](#)
 - getConfigFilePath, [19](#)
 - getInputFilePath, [19](#)
 - getOutputFilePath, [19](#)
 - getPort, [19](#)
 - UserInterface, [18](#)
- what
 - BasicClientException, [11](#)